

Database as a Service: Challenges and Solutions for Privacy and Security

Elena Ferrari
DICOM, University of Insubria
Via Mazzini, 5, 22100 Varese, Italy
elena.ferrari@uninsubria.it

Abstract

This paper analyzes the most relevant privacy and security breaches that may arise in the Database as a Service model. Then, it reviews the state of the art in view of the identified privacy and security requirements. The analysis of the state of the art shows that many open problems still remain to be solved.

1. Introduction

In the era of cloud computing, resources and applications are provided as a service over the Internet [12]. One of the main benefits of such paradigm is the cost reduction, in that users pay only for the services they use and not for their development, installation, and maintenance. Other important benefits of the cloud computing paradigm are in terms of scalability, better quality of service, and more effective allocation of internal resources. Cloud computing is today a growing reality, just to think that Gartner Inc. predicts that by 2012, 80 percent of Fortune 1000 enterprises will pay for some cloud-computing services, while 30 percent of them will pay for cloud-computing infrastructures.¹

In this scenario, an important role is played by data management services, since data are one of the main assets of each organization, from the smallest company to international corporations. In this respect, a new emerging option is represented by the *Database as a Service* (DbaaS) paradigm. DbaaS lays in the field known as Software as a Service (SaaS) and it is the delivery of the typical functionalities of a DBMS in the cloud. The market for this kind of solution is rapidly growing and has attracted the interests of many vendors, from IBM and Microsoft, to small innovators such as EnterpriseDB, LongJump, and Elastra. According to this paradigm, data management moves from a traditional client-server architecture, where the data owner directly manages the DBMS, and answers user queries to

a third party architecture where data owners are no more totally responsible for data management. Rather they outsource their data (or portions of them) to one or more *data service providers* (or *publishers*) that provide data management services.

However, all the benefits of data outsourcing in terms of costs reduction and better services are not enough to make data outsourcing widely adopted. One of the most serious obstacle to the widespread use of DbaaS is related to security [13]. Indeed, outsourcing data to a third party may be perceived by the data owner as a sort of “loss of control” over its data, unless one can have strong guarantees on the security and privacy of the outsourced data.

The challenge is therefore how to ensure properties such as confidentiality, integrity, authenticity, and privacy, even if data are not directly managed by the owner but by a third party. A range of solutions can be adopted, offering different security and privacy guarantees. Another important issue is how to trade-off between efficiency/scalability and security and privacy requirements.

The aim of this paper is to first discuss the main requirements towards a secure and private data outsourcing and then to review the state of the art in view of the identified requirements.

More precisely, the remainder of this paper is organized as follows. Next section presents some background information on privacy and security needed to understand the rest of the paper. Section 3 discusses the main privacy and security requirements related to data outsourcing, both from the data owner and end users point of view. Section 4 reviews the state of the art in view of the identified requirements, whereas Section 5 concludes the paper by discussing future research directions.

2. Background on security and privacy

Traditionally, securing data requires to address three main issues: *data confidentiality* (or secrecy), that is, preventing improper disclosure of information to unauthorized users; *data integrity*, that is, protecting data from unau-

¹www.gartner.com.

thorized or improper modifications or deletions, and *data availability*, that is, prevention and recovery from hardware and software errors and from malicious data denials making the database or some of its portions not available [9].

To deal with such issues any DBMS is equipped with its own security services. One of the most important one is the *access control mechanism* (or *reference monitor*), which is a trusted software module that intercepts each access request submitted to the system (for instance, SQL statements in case of relational DBMSs) and, on the basis of the specified authorizations, it determines whether the access can be partially or totally authorized, or it should be denied. As such, its main purpose is to ensure both data integrity and confidentiality. The authorizations basically state who can access which resource and under which mode. Authorizations are specified according to a set of *access control policies*, that define the high-level rules according to which access control must take place. Whereas, the *administration policies* specify who can grant or revoke authorizations (e.g., the security administrator, resource owners).

In contrast, data privacy relates to an individual's right to determine how, when, and to what extent information about the self will be released to another person or to an organization [14]. There are various definitions of privacy in the literature. Some of them consider confidentiality and privacy as synonyms, while some regard privacy as a different concept. For example, Anderson defined privacy as secrecy for benefit of the individual, whereas confidentiality as secrecy for the benefit of the organization. Alternatively, privacy can also be described by the ability to have control over the collection, storage, access, communication, manipulation and disposition of data. Therefore, it can be said that privacy is a much broader concept than confidentiality. Therefore, privacy protection is based on access control, but ensuring privacy requires something more, such as for instance, verifying that data are used only for the purposes authorized by the user and not for other purposes, or obtaining and recording the user consents. One of the first step toward data privacy protection is therefore the availability of a language to express privacy preferences wrt the treatment of personal data. To this purpose, many privacy policies languages have been proposed so far (e.g., P3P, IBM EPAL) [15]. These languages mainly describe organization's data practices, what information is collected from individuals, what the information will be used for, whether the organization provides access to the information, who the recipients are of any result generated from the information, how long the information will be retained, and who will be informed in the event of a dispute. Users can express his/her preferences in a set of preference-rules to be used to make decisions regarding the acceptability of privacy policies.

3 Security and privacy requirements

It is obvious that in order to make the data outsourcing paradigm well accepted and widespread used it is necessary to fulfill main security/privacy requirements of both data owners and end users. In what follows, we discuss which are the main requirements in this respect. Such requirements greatly depend on the assumptions made on the data service providers. Clearly, the requirements change according to the considered reference architecture. Indeed, a naive solution toward a secure and private data outsourcing is to assume the publisher to be *trusted*, that is, to assume it always operates according to the owner's security/privacy policies. However, making this assumption is not realistic, especially for Web-based systems that can be easily attacked and penetrated. Additionally, verifying that a publisher is trusted is a very costly operation. Therefore, the research is now focusing on the definition of techniques to satisfy main security/privacy properties even in the presence of an *untrusted publisher* that can maliciously modify/delete the data it manages or misuse them (for instance by releasing them to unauthorized users). Therefore, in what follows, we discuss the main security/privacy requirements having in mind such more general scenario. This requires also to devise alternative ways of performing access control wrt the one discussed in Section 2, since the reference monitor cannot be hosted at the publisher side, unless appropriate countermeasures are taken. We categorize the requirements into two main classes: those referring to the data owner and those referring to the end users.

3.1 Data owner

Confidentiality. As we have seen in Section 2, confidentiality means protecting data from unauthorized read operations. Usually, confidentiality requirements are encoded through a set of authorizations (cfr. Section 2), that basically state who can access which portions of the owner data and under which mode. Under the DbaaS model, we can identify two different confidentiality requirements. A first requirement is what we have called in [5] *confidentiality wrt users* – that is, protecting data against unauthorized read operations by end users.

However, in a scenario where data publishers may be untrusted, confidentiality issues are also related to the provider hosting the DBMS. There thus exists a second relevant confidentiality requirement, called *confidentiality wrt publishers*, which deals with protecting owner's data from read operations by publishers.

Integrity. Ensuring data integrity means ensuring a twofold goal. First, in a contest where the data publisher may be untrusted, this implies to verify that data are not altered by the

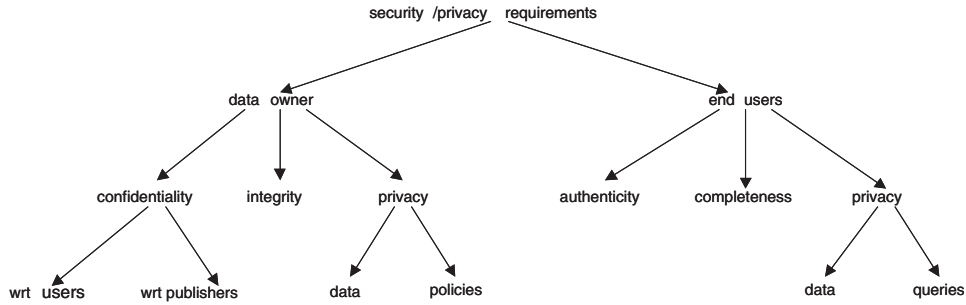


Figure 1. A taxonomy of privacy/security requirements

publisher itself (for instance, by inserting some fake records or maliciously deleting some of the data it manages). Second, it must be verified that data are modified by users according to the owner authorizations.

Privacy. Although some proposals exist of privacy-preserving DBMSs (e.g., [1, 4]), such proposals are mainly conceived for a traditional DBMS setting, as such their extension to cope with the requirements of the DbaaS model is still an open issue. Moreover, a further privacy requirement is related to the owner access control policies. Indeed, the way one protects his/her resources and selects who can access them may result in a leakage of private information. Therefore, information on the defined access control policies should be protected from unauthorized accesses by the publishers.

3.2 End users

Authenticity. Ensuring data authenticity implies that users are assured that the received data come from the intended source. In a context where the data publisher may be untrusted, this implies also to make end users able to verify that data have not been altered by the publisher itself.

Completeness. A further security requirement is related to data availability, that is, to the fact that a user must be supplied by the data service provider with *all* the information he/she is authorized to access on the basis of the stated authorizations. In the context of DbaaS, such requirement is often referred to as *completeness*. Since answers to user's queries are generated by possible untrusted publishers, the completeness requirement ensures that a user receiving a query answer is able to verify that he/she receives all the data 1) answering the submitted query and 2) satisfying owner's access control policies.

Privacy. In addition to the data privacy requirements discussed in Section 3.1, a further privacy requirement is important, especially in the presence of an untrusted publisher,

that is, the protection of user queries. Indeed, an untrusted publisher may misuse the data access patterns revealed by user queries to infer some private information.

Figure 1 graphically summarizes the discussed requirements.

4 State of the Art

In this section, we survey the main research results obtained so far in the field of secure and private data outsourcing, by grouping them according to the security/privacy requirements they address.

4.1 Confidentiality

Most of the effort carried out to address confidentiality in data outsourcing has been focused on confidentiality wrt publishers only [10, 11, 20]. The majority of the proposals enforce confidentiality by means of cryptographic-based solutions. The idea is quite simple. Publishers do not manage clear-text data, rather data are encrypted by the owner before their outsourcing. Since the publisher does not receive any decryption key, it is not able to access the data it manages. Therefore, the publisher should be able to answer user queries over encrypted data. In recent years this problem has been deeply investigated by several researchers that develop solutions for different data models.

For instance, the approach proposed by Hacigumus et al. works for relational data [10, 11] and exploits binning techniques and privacy homomorphic encryption. Binning techniques are used to perform selection queries on encrypted relational data, whereas homomorphic encryption is used to manage aggregate queries over encrypted tuples. To understand how the approach works let us consider selection queries. The basic idea is that for each relation R the owner first divides the domain of each attribute in R into distinguished partitions, to which it assigns a different id. Then, for each tuple t in R , the owner sends the publisher the cor-

responding encrypted tuple t' , together with the ids of the partitions to which t 's attribute values belong to. A user, before submitting a query to a publisher, rewrites it in terms of partition ids. Therefore, the publisher is able to evaluate the query by exploiting the received partition ids. However, by exploiting partition ids the publisher is able to return a user only an approximate result. Therefore, the user receiving an answer by a publisher should refine the received answer. Moreover, it is necessary to trade-off between the precision of the answer to a user query and the information that can be in

To enhance the set of queries that can be performed, the proposed encryption scheme has been extended in [11] with privacy homomorphisms. This class of encryption functions allows one to calculate arithmetic operations directly on encrypted data. This property is exploited to evaluate aggregate queries over encrypted tuples.

Another relevant effort that works for textual data is the one by Song et al. [20]. Given a set of words, the proposed scheme first encrypts each word using a symmetric encryption algorithm with a single secret key k . Then, it generates the XOR of each encrypted word with a pseudo random number. The resulting ciphered words can be then outsourced to the third party. According to this scheme, when a user needs to search for a keyword W , it generates the encrypted word $E_k(W)$ and computes $E_k(W) \oplus S$, where S is the corresponding pseudo random number. This simple scheme allows the third party to search for keyword W in the encrypted data, by simply looking for $E(W) \oplus S$, thus without gaining any information on the clear text. Since occurrences of the same word are xored with different pseudo random numbers, by analyzing the distribution of the encrypted words, no information could be inferred regarding the clear text. Moreover, the scheme proposed in [20] is defined in such a way that users are able to locally compute pseudo random numbers without any interaction with the data owner.

In [5, 6] the above two methods have been combined and extended to support XPath queries over encrypted XML data.

In contrast, few proposals deal with confidentiality wrt users. One of the most notable one addresses the problem in the context of XML data [5, 6]. According to this proposal, as usual, confidentiality requirements wrt users are expressed by the owner through a set of authorizations. Then, data managed by publishers are not encrypted with a single key, as in the previous proposals. Rather, they are encrypted with different keys on the basis of owner's authorizations. Each user then receives from the owner all and only the keys corresponding to data portions he/she can access according to the specified authorizations.²

²How a data portion is defined depends on the considered data model.

The main weak point of this solution is related to key management as it may require the management of a great number of keys. To limit the number of keys that need to be generated, different strategies can be adopted. For instance, in [5] an hierarchical key assignment scheme is used which requires to permanently store, in the worst case, a number of keys linear in the number of specified authorizations.

Note moreover that such cryptographic solutions also provide some privacy guarantees wrt the disclosure of user queries to the publisher, in that the publisher receives and evaluates an encrypted query.

4.2 Authenticity and integrity

Up to now, almost all the research proposals do not consider the problem of integrity requirements arising from data updates at the publisher side, that is, it is the owner that periodically updates the data and send them to the publisher. Users can query the data managed by the publisher but not modify the data it manages. Management of updates at the publisher side is still an open problem. Therefore, in what follows, we do not consider such issue.

An intuitive way of enforcing authenticity wrt the user is through the use of digital signatures. Unfortunately, standard signature techniques are not suitable for third party architectures. This is mainly due to the fact that users must be able to verify authenticity wrt the owner, that is, they must verify that the publisher has not maliciously altered the data it manages. However, due to the authorizations specified by the owner, they may be able to access only a subset of the data they request to the publisher. The challenge is therefore how they can be able to validate the owner's signature even if they receive a selected portion of the signed data. A naive solution is that the owner applies the signature at the finest granularity level, that is, to impose the owner to separately sign each possible data portion. This set of digital signatures could be outsourced together with the data, and could be properly returned to users by the publisher. However, this solution implies an enormous overhead both in owner computation and in query answer size. For this reason, in the recent years, several alternative strategies have been proposed making use of alternative ways for signature generation. All the proposals are based on the delivery of some metadata, often referred to as *verification objects*, to the users in addition to the query answer and the generated signature. Such metadata make the user able to validate the signature even if he/she is authorized to access only a portion of the data over which it has been generated.

Many proposal exploit Merkle trees [17]. Merkle trees have been initially proposed as a method to sign multiple messages, by producing a unique digital signature. To this purpose, a binary hash tree is generated by means of a bottom-up recursive computation. First, for each message

m , a different leaf containing the hash value of m is inserted in the tree; then, the hash value of an internal node is computed as $H(h_l || h_r)$, where $h_l || h_r$ denotes the concatenation of the hash values corresponding to the left and right children nodes, and $H()$ is an hash function. The root node of the resulting binary hash tree is then digitally signed by using a standard signature technique and distributed. A user is able to validate the signature by having a subset of the messages and a set of additional hash values. Indeed, a user, by having hash values of the missing messages, is able to locally build the binary hash tree and thus to validate the signature.

In the context of data outsourcing Devanbu et al. [7] exploited Merkle hash trees to prove the completeness and authenticity of relational query answers. A similar approach has been proposed in [2, 8] for the secure outsourcing of XML data. [19] uses Merkle trees and hashing constructs to prove completeness and authenticity of range selection and projection SQL queries. The method also supports primary key-foreign key join queries and single and multi-attribute range aggregate queries.

Another technique exploited to ensure authenticity in third party architectures is based on signature aggregation. In general, signature aggregation schemes allow one to aggregate into a unique digital signature n distinct signatures generated by n distinct data owners [3]. The validation of this unique digital signature implies the validation of each component signature. Aggregate signature schemes have also been investigated to aggregate into a unique signature n signatures generated by the same owner. This last kind of aggregation scheme can be adopted to ensure authenticity in third party scenarios. Indeed, according to this solution, the data owner could generate a distinct signature for each distinct data portion and then aggregate them into a unique digital signature. By having only the aggregate signature and by simply validating it, a user is able to authenticate selected messages received by third party. Such kind of solution has been proposed by Mykletun et al. in [16] for relational data.

4.3 Completeness

Completeness is a property that is receiving a growing attention due to its relevance in data outsourcing. Completeness for relational data has been investigated in conjunction with authenticity requirements (cfr. the discussion above). Some of the proposed methods exploit Merkle trees, whereas others exploit aggregate signature schemes.

To illustrate the idea behind them, in what follows we consider two of them which have been developed for range queries. The approach presented in [7] makes use of Merkle trees and assumes that Merkle hash trees are built over tuples sorted according to the attribute a over the range

query's predicate is specified. To make the user able to verify answer's completeness, the third party inserts in the answer two additional tuples, that is, the tuple preceding (i.e., subsequent) the lower (i.e., upper) bound of the result set answering the range query. By having these two values, the user can locally verify that the query answer is complete, that is, that no tuples answering the range query have been omitted. Indeed, by verifying the owner's signature, which has been generated on a Merkle hash tree built over sorted tuples, the user also verifies that the received lower (i.e., upper) tuple effectively precedes (i.e., follows) the ones in the result set.

In contrast, the approach presented in [18] modifies an aggregation signature scheme, thus to include in the signature of a tuple t , the hash value of tuples preceding t according to all possible sortings defined on the relation's attributes. Then, the third party inserts into the range query's answer the boundary tuples, i.e., the tuples preceding the upper and lower bound of the result set, as well as their aggregated signatures. By means of this information, a user is able to prove that the third party has not omitted any tuple.

5. Conclusions

In the era of cloud computing data outsourcing is becoming a vital need. However, in order to be widely accepted and adopted it is mandatory that both users and data owners have strong security and privacy guarantees on the managed data. In this paper, we have surveyed the main privacy and security requirements related to data outsourcing, then we have reviewed the state of the art in view of the identified requirements.

However, many problems still remains open. First of all, most of the research proposals appeared so far exploit cryptographic techniques to achieve privacy and security. One important issue is therefore how to efficiently deal with key management. Moreover, providing authenticity, completeness and confidentiality for any kind of query is still an open issue, since almost all the proposals put restrictions on the kinds of query they support. Another issue that should be investigated is the management of updates at the publisher side, that should be driven by the access control policies specified by the owner. Further, a more deep investigation of the privacy breaches arising from the queries submitted to the publisher is still needed. Indeed, even if queries are encrypted, by inspecting multiple encrypted queries and linking them with public data, the publisher may be able to infer some private information about the user.

Acknowledgments

The work reported in this paper is partially funded by the Italian MIUR under the ANONIMO project (PRIN-2007F9437X).

References

- [1] R. Agrawal, J. Kiernan, R. Srikant, Y. Xu. Hippocratic Databases. In Proc. of the *28th International Conference on Very Large Databases (VLDB'02)*, Hong Kong, China, 143–157, 2002.
- [2] E. Bertino, B. Carminati, E. Ferrari, B. Thuraisingham, A. Gupta. Selective and Authentic Third-Party Distribution of XML Documents. *IEEE Transactions on Knowledge and Data Engineering*, 16(10):1263–1278, 2004.
- [3] D. Boneh, C. Gentry, B. Lynn, H. Shacham. Aggregate and Verifiably Encrypted Signatures from Bilinear Maps. Proc. of *EUROCRYPT 2003: International Conference on the Theory and Applications of Cryptographic Techniques*, Warsaw, Poland, 2003.
- [4] J.W. Byun, N. Li. Purpose based Access Control for Privacy Protection in Relational Database Systems. *VLDB Journal*, 17(4):603–619, 2008.
- [5] B. Carminati, E. Ferrari, E. Bertino. Securing XML Data in Third-Party Distribution Systems. In Proc. of the *14th ACM Conference on Information and Knowledge Management (CIKM'05)*, Bremen, Germany, 99–106, 2005.
- [6] B. Carminati, E. Ferrari. Confidentiality Enforcement for XML Outsourced Data. In Proc. of the *2nd EDBT Workshop on Database Technologies for Handling XML Information on the Web (DATA'06)*, Munich, Germany, 2006.
- [7] P. Devanbu, M. Gertz, C. Martel, S.G. Stubblebine. Authentic Third-party Data Publication. In Proc. of the *14th Annual IFIP WG 11.3 Working Conference on Database Security*, Schoorl, the Netherlands, 2000.
- [8] P. Devanbu, M. Gertz, A. Kwong, C. Martel, G. Nuckolls, S.G. Stubblebine. Flexible Authentication of XML Documents. In Proc. of the *8th ACM Conference on Computer and Communications Security (CCS'01)*, Philadelphia, PA, 2001.
- [9] E. Ferrari. Database Security. In L. Liu, M.T. Ozsu editors, *Encyclopedia of Database Systems (EDS)*, Springer, 2009.
- [10] H. Hacigumus, B. Iyer, C. Li, S. Mehrotra. Executing SQL over Encrypted Data in the Database Service Provider Model. In Proc. of the *ACM SIGMOD'2002 Conference*, Madison, Wisconsin, 216–227, 2002.
- [11] H. Hacigumus, B. Iyer, C. Li, S. Mehrotra. Efficient Execution of Aggregation Queries over Encrypted Relational Databases. In Proc. of the *9th International Conference on Database Systems for Advanced Applications (DASFAA'04)*, Jeju Island, Korea, 125–136, 2004.
- [12] B. Hayes. Cloud Computing. *Communications of the ACM*, 51(7):9–11, 2008.
- [13] J. Heiser, M. Nicolett. Assessing the Security Risks of Cloud Computing. Gartner report, www.gartner.com, June 2008.
- [14] P. C. K. Hung and V. S. Y. Cheng. Privacy. In L. Liu, M.T. Ozsu editors, *Encyclopedia of Database Systems (EDS)*, Springer, 2009.
- [15] P. Kumaraguru, L. Cranor, J. Lobo and S. Calo. A Survey of Privacy Policy Languages. In Proc. of the *Workshop on Usable IT Security Management (USM'07)*, Pittsburgh, PA, 2007.
- [16] E. Mykletun, M. Narasimha, G. Tsudik. Authentication and Integrity in Outsourced Databases. *ACM Transactions on Storage*, 2(2):107–138, 2006.
- [17] R.C. Merkle. A Certified Digital Signature. In Proc. of *Advances in Cryptology*, Santa Barbara, CA, 218–238, 1989.
- [18] M. Narasimha, G. Tsudik. DSAC: Integrity of Outsourced Databases with Signature Aggregation and Chaining. In Proc. of the ACM Conference on Information and Knowledge Management (CIKM'05), Bremen, Germany, 235–236, 2005.
- [19] H. Pang, K.L. Tan. Verifying Completeness of Relational Query Answers from Online Servers. *ACM Trans. Inf. Syst. Secur.* 11(2), 2008.
- [20] D. X. Song, D. Wagner, A. Perrig. Practical Techniques for Searches on Encrypted Data. In Proc. of the *IEEE Symposium on Security and Privacy*, Oakland, CA, 44–55, 2000.